

LAB 7. MODEL COMPRESSION

KNOWLEDGE DISTILLATION, QUANTIZATION AND TRADE-OFF ANALYSIS

ThS. Lê Thị Thùy Trang

2026-04-27

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Ở bài lab trước, sinh viên đã đưa mô hình từ PyTorch sang ONNX, kiểm tra consistency test và benchmark latency. Đó là bước rất quan trọng để mô hình rời khỏi notebook và tiến gần hơn đến triển khai thực tế. Tuy nhiên, một câu hỏi mới sẽ xuất hiện ngay sau đó:

Mô hình chạy được rồi, nhưng có quá nặng không? Có chạy đủ nhanh không? Có cần nén lại không?

Trong các hệ thống AI thật, đặc biệt là các hệ thống có yếu tố thời gian thực như Smart Campus, camera giám sát, thiết bị edge hoặc server CPU, mô hình không chỉ cần đúng mà còn cần **gọn, nhanh** và **đủ ổn định**. Một mô hình lớn có thể có accuracy cao, nhưng nếu inference quá chậm hoặc file model quá nặng, việc triển khai sẽ gặp khó khăn.

Bài lab này giúp sinh viên làm quen với tư duy **model compression**. Cụ thể, sinh viên sẽ chọn ít nhất một trong hai hướng:

- **Quantization**: giảm độ chính xác số học của trọng số/mô hình, ví dụ từ FP32 xuống INT8;
- **Knowledge Distillation**: dùng một mô hình lớn làm teacher để huấn luyện một mô hình nhỏ hơn làm student.

Điểm quan trọng nhất của bài lab này không phải là “nén cho bằng được”, mà là biết phân tích trade-off:

Accuracy ↔ Latency ↔ Model Size

Nếu nén xong mà mô hình nhỏ hơn nhưng accuracy tụt quá nhiều thì chưa chắc đã đáng. Nếu accuracy gần như giữ nguyên nhưng latency không cải thiện thì cũng cần đặt câu hỏi. Nếu model nhỏ đi rất nhiều và chạy nhanh hơn một chút, có thể vẫn đáng trong môi trường triển khai hạn chế bộ nhớ.

Nói ngắn gọn, bài lab này rèn cho sinh viên thói quen:

Không chỉ hỏi mô hình có đúng không, mà còn hỏi mô hình có đáng để triển khai không.

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo scaffold đã được hoàn thiện đủ để chạy ít nhất một kỹ thuật nén;
2. Một baseline model từ lab trước, có thể là ONNX model hoặc PyTorch checkpoint;
3. Một compressed model, ví dụ:
 - `vit_smartcampus_dynamic_int8.onnx` nếu chọn quantization;
 - `student_best.pt` nếu chọn knowledge distillation;

4. Kết quả đánh giá trước và sau nén: accuracy, macro-F1;
5. Kết quả benchmark trước và sau nén: latency, p95 latency, throughput, model size;
6. Một bảng trade-off tổng hợp;
7. Một báo cáo ngắn giải thích kết quả thay vì chỉ dán số liệu;
8. Nếu chọn KD, cần có W&B run hoặc minh chứng quá trình huấn luyện student model;
9. Nhận xét được khi nào nên chọn quantization, khi nào nên chọn KD.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code đã hoàn thiện.
2. File báo cáo theo mẫu `REPORT_TEMPLATE.md` hoặc báo cáo riêng.
3. Kết quả đánh giá baseline, ví dụ `eval_baseline_onnx.json`.
4. Kết quả đánh giá model sau nén, ví dụ `eval_quantized_onnx.json`.
5. Kết quả benchmark trước và sau nén, ví dụ `benchmark_quantization.csv`.
6. Bảng trade-off: `tradeoff_table.csv` và/hoặc `tradeoff_table.md`.
7. Link W&B nếu chọn Knowledge Distillation.
8. Link lưu model/checkpoint nếu file quá lớn và không commit trực tiếp lên GitHub.
9. Nhận xét ngắn: nén xong có đáng không, căn cứ vào số liệu nào.

Lưu ý quan trọng: không commit dataset, checkpoint lớn hoặc file ONNX lớn lên GitHub. Nếu file lớn, sinh viên cần lưu ở Google Drive, OneDrive hoặc nơi lưu trữ phù hợp, sau đó ghi link trong báo cáo.

1.4 Bối cảnh bài toán

Chúng ta tiếp tục dùng case study:

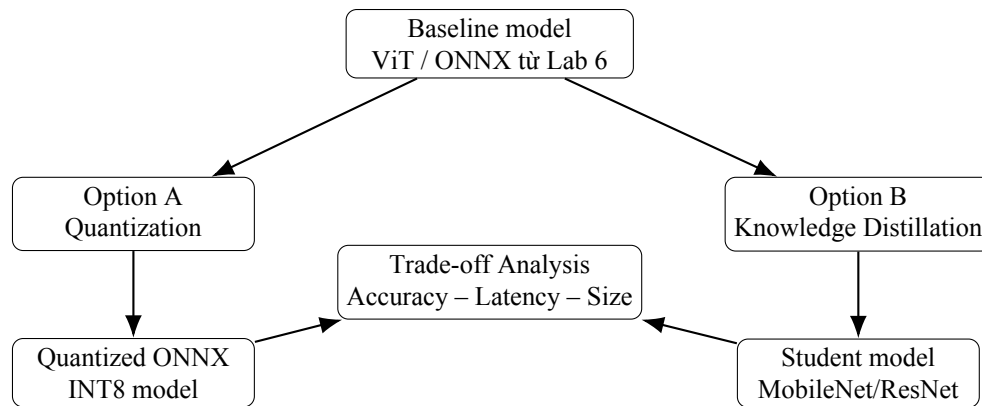
Smart Campus Scene Classification

Mô hình nhận ảnh không gian trong trường học và phân loại vào một trong năm lớp:

- classroom;
- computerroom;
- library;
- corridor;
- office.

Ở các bài trước, mô hình Vision Transformer đã được huấn luyện, sau đó được export sang ONNX và benchmark. Bài lab này giả sử mô hình baseline đã có. Bây giờ, ta muốn làm mô hình gọn hơn hoặc nhanh hơn.

Có hai con đường chính:



Nếu lab trước giống như đưa mô hình ra chạy thử trên đường, thì lab này giống như hỏi: có cần làm chiếc xe nhẹ hơn, tiết kiệm hơn, chạy nhanh hơn không? Và nếu làm nhẹ đi thì có mất độ an toàn không?

1.5 Cấu trúc repo scaffold

Repo scaffold của bài lab có cấu trúc như sau:

```

csc4005_lab7_compression_kd_quantization_scaffold/
README.md
REPORT_TEMPLATE.md
requirements.txt
configs/
  quantization_dynamic.json
  kd_student_mobilenet.json
  tradeoff_eval.json
docs/
  LAB_GUIDE_LAB7_COMPRESSION.md
  QUANTIZATION_GUIDE.md
  KD_GUIDE.md
  TRADEOFF_ANALYSIS_GUIDE.md
  RUBRIC.md
notebooks/
  lab7_compression_demo.ipynb
src/
  dataset.py
  models.py
  metrics.py
  runtime.py
  evaluate_pytorch.py
  evaluate_onnx.py
  quantize_onnx.py
  kd_train_student.py
  benchmark.py
  make_tradeoff_table.py
  utils.py
ci/
  check_structure.py
  smoke_imports.py
data/
checkpoints/

```

```
models/  
outputs/  
.github/workflows/  
  ci.yml
```

Các file quan trọng nhất:

- `src/quantize_onnx.py`: tạo ONNX model đã quantize;
- `src/kd_train_student.py`: scaffold huấn luyện student model bằng KD;
- `src/evaluate_onnx.py`: đánh giá ONNX model;
- `src/evaluate_pytorch.py`: đánh giá PyTorch student model;
- `src/benchmark.py`: benchmark latency, throughput, model size;
- `src/make_tradeoff_table.py`: tạo bảng trade-off;
- `REPORT_TEMPLATE.md`: mẫu báo cáo;
- `RUBRIC_LAB7.md`: rubric chấm điểm.

2 Chuẩn bị môi trường thực hành

2.1 Điều kiện cần

Sinh viên cần chuẩn bị:

- Python 3.10 hoặc phiên bản tương thích;
- Git;
- Repo GitHub Classroom hoặc repo cá nhân;
- Dataset MIT Indoor Scenes 67 subset 5 lớp;
- File ONNX từ Lab 6 nếu chọn quantization;
- Checkpoint PyTorch teacher nếu chọn Knowledge Distillation;
- Tài khoản W&B nếu chọn KD;
- Máy tính có đủ RAM để chạy inference/huấn luyện student model.

Bài lab này có thể chạy trên CPU nếu chọn quantization. Nếu chọn Knowledge Distillation, GPU sẽ giúp quá trình train student model nhanh hơn, nhưng không bắt buộc nếu giảm số epoch và batch size.

2.2 Clone repo

Sau khi nhận link GitHub Classroom, clone repo:

```
git clone <student-repo-url>  
cd csc4005_lab7_compression_kd_quantization_scaffold
```

Kiểm tra cấu trúc repo:

```
python ci/check_structure.py
```

Nếu thấy:

```
Structure check passed.
```

thì cấu trúc cơ bản đã ổn.

2.3 Tạo environment

Trên macOS hoặc Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
```

Trên Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install --upgrade pip
```

2.4 Cài thư viện

```
pip install -r requirements.txt
```

Các thư viện chính:

- torch, torchvision: dùng cho KD và student model;
- onnx, onnxruntime: dùng cho quantization và inference ONNX;
- onnxruntime.quantization: dùng cho ONNX dynamic quantization;
- pandas, numpy: xử lý bảng benchmark và trade-off;
- wandb: log thí nghiệm nếu chọn KD;
- scikit-learn: tính accuracy và macro-F1.

2.5 Chuẩn bị dữ liệu

Dữ liệu nên có cấu trúc:

```
data/mit_indoor_smartcampus_5/
classroom/
computerroom/
library/
corridor/
office/
```

Nếu dữ liệu nằm ngoài repo, sinh viên có thể truyền đường dẫn tuyệt đối vào tham số `--data_dir`.

Ví dụ:

```
python -m src.evaluate_onnx \
--onnx_path models/vit_smartcampus.onnx \
--data_dir /duong_dan/mit_indoor_smartcampus_5 \
--output_json outputs/eval_baseline_onnx.json
```

2.6 Chuẩn bị model đầu vào

2.6.1 Nếu chọn Quantization

Cần có file ONNX từ Lab 6:

```
models/vit_smartcampus.onnx
```

Nếu file ONNX để ở nơi khác, truyền đúng đường dẫn bằng `--input_onnx`.

2.6.2 Nếu chọn Knowledge Distillation

Cần có checkpoint teacher:

```
checkpoints/teacher_vit_best_model.pt
```

Teacher thường là ViT đã được fine-tune ở lab trước. Student trong scaffold có thể là:

- `mobilenet_v2`;
- `resnet18`.

2.7 Nhắc lại quy tắc không push file lớn

Các thư mục/file lớn đã được đưa vào `.gitignore`:

```
data/  
checkpoints/  
models/  
outputs/  
*.pt  
*.pth  
*.onnx
```

Repo GitHub nên chứa code, hướng dẫn chạy, báo cáo và kết quả nhỏ. Dataset, checkpoint và ONNX lớn nên lưu bên ngoài.

3 Voila, hướng dẫn thực hành ở đây

3.1 Bước 0. Chọn chiến lược làm bài

Sinh viên cần chọn ít nhất một trong hai hướng:

- **Hướng A – Quantization**: nhanh hơn, phù hợp nếu đã có ONNX model từ Lab 6;
- **Hướng B – Knowledge Distillation**: sâu hơn, cần train student model, phù hợp nếu muốn hiểu compression ở mức mô hình.

Nếu còn phân vân, gợi ý chọn như sau:

- Máy yếu, muốn tập trung vào deployment: chọn Quantization;
- Muốn thử train model nhỏ hơn, có GPU hoặc kiên nhẫn chờ: chọn KD;
- Nhóm khá: làm cả hai rồi so sánh.

Yêu cầu tối thiểu: làm được một hướng và có bảng trade-off.
Yêu cầu tốt: làm cả Quantization và KD, sau đó so sánh hai kỹ thuật.

4 Hướng A. Quantization

4.1 A1. Kiểm tra baseline ONNX

Trước tiên, cần có file baseline ONNX:

```
models/vit_smartcampus.onnx
```

Đánh giá baseline ONNX:

```
python -m src.evaluate_onnx \
  --onnx_path models/vit_smartcampus.onnx \
  --data_dir data/mit_indoor_smartcampus_5 \
  --output_json outputs/eval_baseline_onnx.json
```

Sau khi chạy, kiểm tra file:

```
outputs/eval_baseline_onnx.json
```

File này nên có các thông tin như:

```
{
  "accuracy": 0.82,
  "macro_f1": 0.81,
  "model_size_mb": 330.5,
  "num_samples": 500
}
```

Các con số trên chỉ là minh họa. Sinh viên cần đọc kết quả thật của mình.

4.2 A2. Quantize ONNX model

Chạy dynamic quantization:

```
python -m src.quantize_onnx \
  --input_onnx models/vit_smartcampus.onnx \
  --output_onnx models/vit_smartcampus_dynamic_int8.onnx \
  --mode dynamic
```

Sau khi chạy, kiểm tra:

```
models/vit_smartcampus_dynamic_int8.onnx
models/quantization_report.json
```

File quantization_report.json cho biết model giảm kích thước bao nhiêu.

Ví dụ:

```
{
  "method": "onnx_dynamic_quantization",
  "baseline_size_mb": 330.5,
  "compressed_size_mb": 84.2,
  "size_reduction_percent": 74.5
}
```

Nếu model nhỏ đi nhiều, đó là tín hiệu tốt. Nhưng xin đừng vội reo vui quá sớm. Nhỏ hơn chưa chắc đã tốt hơn nếu accuracy tụt mạnh hoặc latency không cải thiện.

4.3 A3. Đánh giá model sau quantization

Chạy:

```
python -m src.evaluate_onnx \
--onnx_path models/vit_smartcampus_dynamic_int8.onnx \
--data_dir data/mit_indoor_smartcampus_5 \
--output_json outputs/eval_quantized_onnx.json
```

So sánh hai file:

```
outputs/eval_baseline_onnx.json
outputs/eval_quantized_onnx.json
```

Câu hỏi cần trả lời:

1. Accuracy có giảm không?
2. Macro-F1 có giảm không?
3. Nếu giảm, giảm bao nhiêu điểm phần trăm?
4. Mức giảm này có chấp nhận được trong bài toán Smart Campus không?

4.4 A4. Benchmark baseline và quantized model

Chạy benchmark:

```
python -m src.benchmark \
--onnx_paths models/vit_smartcampus.onnx models/vit_smartcampus_dynamic_int8.onnx \
--names baseline_onnx quantized_int8 \
--batch_sizes 1 4 8 \
--warmup 10 \
--repeat 50 \
--output_csv outputs/benchmark_quantization.csv
```

Output:

```
outputs/benchmark_quantization.csv
```

4.5 Nếu nhìn bảng benchmark như nhìn bức vách thì không sao hết, đọc tiếp là ổn áp ngay

Các cột cần đọc:

- model: baseline hay quantized;
- batch_size: số ảnh mỗi lượt inference;
- mean_latency_ms: thời gian trung bình cho một lượt inference;
- p95_latency_ms: mốc latency mà 95% lượt chạy nhanh hơn hoặc bằng;
- throughput_img_per_sec: số ảnh xử lý được trong một giây;
- model_size_mb: kích thước file model.

Một kết luận tốt không chỉ nói:

Model quantized nhẹ hơn.

Mà nên nói:

Model quantized nhẹ hơn khoảng 70%, latency batch size 1 giảm khoảng 20%, nhưng macro-F1 giảm 1.5 điểm. Với yêu cầu triển khai CPU, trade-off này có thể chấp nhận được.

5 Hướng B. Knowledge Distillation

5.1 B1. Ý tưởng KD

Knowledge Distillation dùng một mô hình lớn làm teacher để hướng dẫn một mô hình nhỏ hơn.

Trong bài lab này:

Teacher: ViT từ lab trước

Student: MobileNetV2 hoặc ResNet18

Teacher có thể mạnh nhưng nặng. Student nhỏ hơn, chạy nhanh hơn, nhưng nếu train trực tiếp bằng label thật thì có thể học chưa tốt. KD giúp student học thêm “soft knowledge” từ teacher.

Loss thường có dạng:

```
loss = alpha * CE(student_logits, labels)
      + (1 - alpha) * KD_loss(student_logits, teacher_logits, T)
```

Trong đó:

- α : cân bằng giữa label thật và tín hiệu từ teacher;
- T : temperature, làm mềm phân phối xác suất;
- CE: cross entropy với nhãn thật;
- KD_loss : thường dùng KL divergence giữa phân phối teacher và student.

5.2 B2. Đăng nhập W&B

Nếu chọn KD, sinh viên cần dùng W&B để log quá trình train:

```
wandb login
```

Project gợi ý:

```
csc4005-lab7-compression
```

5.3 B3. Train student bằng KD

Chạy:

```
python -m src.kd_train_student \
  --teacher_checkpoint checkpoints/teacher_vit_best_model.pt \
  --data_dir data/mit_indoor_smartcampus_5 \
  --student_model mobilenet_v2 \
  --alpha 0.5 \
  --temperature 4.0 \
```

```
--epochs 10 \  
--batch_size 16 \  
--use_wandb
```

Output kỳ vọng:

```
outputs/kd_student/student_best.pt  
outputs/kd_student/kd_summary.json
```

Trong scaffold, một số phần được để theo tinh thần mở để sinh viên đọc, hiểu và cải thiện. Sinh viên có thể chỉnh:

- alpha;
- temperature;
- student_model;
- epochs;
- batch_size;
- learning rate.

5.4 B4. Đánh giá student model

Chạy:

```
python -m src.evaluate_pytorch \  
--checkpoint outputs/kd_student/student_best.pt \  
--student_model mobilenet_v2 \  
--data_dir data/mit_indoor_smartcampus_5 \  
--output_json outputs/eval_kd_student.json
```

Kết quả cần có:

```
accuracy  
macro_f1  
model_size_mb  
num_samples
```

5.5 B5. Benchmark student model

Trong scaffold hiện tại, benchmark chính tập trung vào ONNX model. Nếu muốn benchmark student PyTorch đầy đủ, sinh viên có thể mở rộng thêm. Ở mức tối thiểu, sinh viên cần ghi nhận model size và đánh giá metric của student. Ở mức tốt hơn, sinh viên export student sang ONNX rồi benchmark tương tự hướng Quantization.

Gợi ý mở rộng:

```
student_best.pt → export student ONNX → benchmark student ONNX
```

Đây là phần cộng điểm tốt nếu nhóm muốn làm sâu hơn.

6 Bước tạo bảng trade-off

Sau khi có kết quả đánh giá và benchmark, sinh viên tạo bảng trade-off.

Ví dụ với hướng Quantization:

```
python -m src.make_tradeoff_table \
--baseline_eval outputs/eval_baseline_onnx.json \
--compressed_eval outputs/eval_quantized_onnx.json \
--baseline_benchmark outputs/benchmark_quantization.csv \
--compressed_benchmark outputs/benchmark_quantization.csv \
--output_csv outputs/tradeoff_table.csv \
--output_md outputs/tradeoff_table.md
```

Tùy cấu trúc benchmark, sinh viên có thể cần tách riêng file benchmark baseline và compressed. Nếu dùng chung một file, hãy đảm bảo báo cáo trình bày rõ dòng nào là baseline, dòng nào là compressed.

Bảng tối thiểu cần có:

Model	Accuracy	Macro-F1	Mean latency @bs=1	Throughput @bs=1	Size	Nhận xét
Baseline	...	...	...	...	...	...
Com-pressed	...	...	...	...	...	...

6.1 Bảng trade-off không phải để trang trí

Bảng trade-off là linh hồn của bài lab này. Sinh viên không nên chỉ tạo bảng rồi để đó. Cần đọc bảng và trả lời:

1. Model sau nén nhỏ hơn bao nhiêu phần trăm?
2. Latency giảm hay tăng?
3. Throughput tăng hay giảm?
4. Accuracy giảm bao nhiêu?
5. Macro-F1 giảm bao nhiêu?
6. Nếu triển khai thật, có chọn model sau nén không?

Ví dụ nhận xét tốt:

Sau dynamic quantization, model size giảm từ 330 MB xuống 85 MB, tương đương giảm khoảng 74%. Ở batch size 1, latency giảm từ 90 ms xuống 70 ms. Accuracy giảm từ 0.84 xuống 0.83 và macro-F1 giảm 0.01. Với bối cảnh triển khai CPU trong hệ thống Smart Campus, trade-off này có thể chấp nhận được.

Ví dụ nhận xét chưa tốt:

Model sau nén tốt hơn vì nhẹ hơn.

Nhận xét này chưa đủ vì không nói rõ nhẹ hơn bao nhiêu, accuracy có giảm không, latency có cải thiện không, và có phù hợp với bối cảnh triển khai không.

7 Khi nào chọn Quantization, khi nào chọn KD?

7.1 Quantization phù hợp khi nào?

Quantization thường phù hợp khi:

- đã có model train tốt;
- muốn giảm kích thước model nhanh;

- không muốn train lại nhiều;
- muốn tối ưu inference trên CPU;
- chấp nhận kiểm tra lại accuracy sau nén.

Nhược điểm là không phải lúc nào latency cũng giảm mạnh. Có thể model nhỏ hơn nhưng tốc độ chỉ cải thiện ít, tùy runtime và phần cứng.

7.2 KD phù hợp khi nào?

Knowledge Distillation phù hợp khi:

- teacher model quá lớn;
- muốn có student model nhỏ hơn hẳn;
- sẵn sàng train lại;
- muốn triển khai trên thiết bị hạn chế tài nguyên;
- có đủ dữ liệu để student học lại.

Nhược điểm là phức tạp hơn. Cần chọn student architecture, temperature, alpha, epoch, learning rate và theo dõi quá trình train.

7.3 Nếu phải chọn cho Smart Campus?

Không có câu trả lời duy nhất. Nhưng có thể suy nghĩ như sau:

- Nếu cần giải pháp nhanh để giảm kích thước file ONNX: chọn Quantization;
- Nếu cần model thật sự nhỏ để chạy trên edge device: thử KD;
- Nếu yêu cầu accuracy rất cao và latency chưa quá căng: có thể giữ baseline;
- Nếu model lớn gây khó triển khai: phải chấp nhận trade-off và chọn phương án cân bằng.

8 Những lỗi rất hay gặp

8.1 Lỗi 1. Không có baseline từ Lab 6

Nếu không có baseline ONNX hoặc checkpoint teacher, sinh viên sẽ không có điểm xuất phát để nén. Hãy kiểm tra:

```
models/vit_smartcampus.onnx
checkpoints/teacher_vit_best_model.pt
```

Ít nhất cần một trong hai, tùy hướng làm bài.

8.2 Lỗi 2. Nghĩ rằng model nhỏ hơn nghĩa là tốt hơn

Model nhỏ hơn là một tín hiệu tốt, nhưng chưa đủ. Nếu accuracy tụt mạnh, compressed model có thể không đáng dùng.

Cần nhìn đủ ba yếu tố:

```
accuracy
latency
size
```

8.3 Lỗi 3. Benchmark baseline và compressed trong điều kiện khác nhau

Nếu baseline chạy batch size 1 còn compressed chạy batch size 8, hoặc một bên chạy khi máy đang rất bận còn bên kia không, kết quả sẽ không công bằng.

Cần đảm bảo:

- cùng máy;
- cùng batch size;
- cùng input size;
- cùng warm-up;
- cùng số lần repeat.

8.4 Lỗi 4. Quên đo lại accuracy sau quantization

Quantization có thể làm output thay đổi. Vì vậy, sau khi quantize xong phải đánh giá lại. Không được chỉ nộp file ONNX đã nén.

8.5 Lỗi 5. KD nhưng không log quá trình train

Nếu chọn KD, cần log quá trình train bằng W&B hoặc ít nhất lưu history rõ ràng. Nếu chỉ nộp checkpoint student mà không có log, rất khó kiểm chứng quá trình huấn luyện.

8.6 Lỗi 6. Chọn temperature và alpha nhưng không giải thích

Trong KD, temperature và alpha không phải con số trang trí. Sinh viên cần giải thích ngắn:

- temperature ảnh hưởng thế nào đến soft label;
- alpha cân bằng nhận thật và tín hiệu từ teacher như thế nào.

8.7 Lỗi 7. So sánh teacher và student không công bằng

Teacher và student cần được đánh giá trên cùng tập dữ liệu, cùng preprocessing và cùng metric. Nếu không, kết luận sẽ thiếu tin cậy.

8.8 Lỗi 8. Push file lớn lên GitHub

Nếu push file .onnx, .pt hoặc dataset lớn, GitHub có thể báo lỗi. Cách tốt hơn là lưu file lớn bên ngoài và ghi link trong báo cáo.

9 Câu hỏi tự kiểm tra sau lab

Sinh viên cần tự trả lời các câu hỏi sau trước khi nộp bài:

1. Model compression là gì?
2. Vì sao một mô hình accuracy cao vẫn có thể khó triển khai?
3. Quantization làm giảm cái gì trong mô hình?
4. Dynamic quantization khác gì với static quantization?
5. Sau quantization, vì sao cần đánh giá lại accuracy?
6. Knowledge Distillation là gì?
7. Teacher model và student model khác nhau như thế nào?
8. Temperature trong KD có vai trò gì?
9. Alpha trong KD có vai trò gì?
10. Accuracy, latency và model size có thể trade-off với nhau như thế nào?
11. Nếu model sau nén nhẹ hơn nhưng accuracy giảm mạnh, có nên dùng không?
12. Nếu model sau nén nhẹ hơn nhưng latency không giảm, có nên dùng không?
13. Trong hệ thống Smart Campus thật, bạn ưu tiên accuracy, latency hay model size? Vì sao?

10 Checklist trước khi nộp bài

Trước khi nộp, sinh viên kiểm tra:

- Repo có chạy được không?
- Có baseline model từ Lab 6 không?
- Có ít nhất một compressed model không?
- Có file đánh giá baseline không?
- Có file đánh giá compressed model không?
- Có benchmark trước/sau nén không?
- Có bảng trade-off không?
- Có phân tích accuracy-latency-size không?
- Nếu chọn KD, có W&B hoặc log train không?
- Có báo cáo theo mẫu không?
- Có tránh push file lớn lên GitHub không?

Nếu tất cả đều ổn, xin chúc mừng: bạn đã đi qua một bước rất quan trọng của triển khai mô hình học sâu. Mô hình không chỉ biết dự đoán, mà còn bắt đầu biết “sống gọn gàng” hơn trong điều kiện triển khai thực tế.